

Attention Is All You Need - Beginner Friendly Study Notes

These notes are based on our conversation explaining the paper section by section in simple language. They are intended as revision notes rather than a verbatim copy of the paper.

Part 1: Why Transformers?

Before Transformers, models like RNNs and LSTMs processed words one at a time. This made training slow and caused difficulty remembering information from far earlier in a sentence. The paper's key idea is that every word should be able to directly look at every other word using Attention.

Introduction & Background

The paper proposes the Transformer, a model built entirely on attention mechanisms instead of recurrence (RNNs) or convolutions (CNNs). This allows parallel processing and better handling of long-range dependencies.

Transformer Architecture

The architecture has an Encoder and a Decoder. The encoder understands the input sentence. The decoder generates the output sentence one token at a time.

Encoder

The encoder contains 6 identical layers. Each layer has: (1) Multi-Head Self-Attention (2) Feed Forward Network. Residual connections and Layer Normalization are applied after each sublayer.

Decoder

The decoder also has 6 layers. Each contains Masked Self-Attention, Encoder-Decoder Attention, and a Feed Forward Network. Masking prevents the model from seeing future words.

Self-Attention

Every word asks: 'Which other words are important for understanding me?' Each word attends to every other word in the same sentence.

Query, Key, Value

Query = What information am I looking for?

Key = What information do I offer?

Value = The actual information I provide.

Queries compare with Keys to decide how much of each Value should be used.

Scaled Dot-Product Attention

Formula:

$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Steps:

1. Compare Query with every Key.
2. Compute similarity scores.
3. Divide by $\sqrt{d_k}$ to stabilize training.
4. Apply Softmax to obtain attention weights.
5. Compute a weighted sum of the Values.

Multi-Head Attention

Instead of one attention mechanism, the model uses multiple heads (8 in the original paper). Each head learns different relationships such as grammar, context, syntax, or long-distance

dependencies. Their outputs are concatenated.

Feed Forward Network

After attention, every token passes independently through the same small neural network: Linear -> ReLU -> Linear.

Residual Connections

Shortcut connections add the input back to the output of each sublayer. This helps gradients flow through deep networks.

Layer Normalization

Normalizes activations after residual addition to make training more stable.

Positional Encoding

Because the Transformer processes all words simultaneously, it needs position information. The paper uses sinusoidal positional encodings based on sine and cosine waves.

Why Self-Attention?

Advantages:

- Parallel computation
- Better long-range dependency modeling
- Shorter information paths between words
- Faster training than RNN-based models

Training

The authors trained on WMT English-German and English-French translation datasets using Adam optimizer, learning-rate warmup, dropout, and label smoothing.

Results

The Transformer achieved state-of-the-art BLEU scores while requiring significantly less training time than previous models.

Experiments

The paper evaluated different numbers of attention heads, model sizes, dropout values, and positional encodings. Multi-head attention and larger models generally performed better.

Attention Visualizations

Different attention heads naturally specialized in different linguistic tasks, such as pronoun resolution and long-distance grammatical relationships.

Final Takeaways

1. Attention lets every word focus on relevant words.
2. Self-Attention replaces recurrence.
3. Query-Key-Value form the core of attention.
4. Scaling and Softmax create stable attention weights.
5. Multi-Head Attention learns multiple relationships.
6. Positional Encoding preserves word order.
7. Residual Connections and LayerNorm stabilize deep learning.
8. This paper introduced the Transformer, which became the foundation of modern LLMs.